VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



OPERATING SYSTEMS (CO2017)

Group Assignment – Week 3

Multi-threading Best Practices via Monte Carlo π Estimation

Instructor: [TODO: instructor name]

Class / Group: [TODO: class] / [TODO: group]

Group members: [TODO: full name] – [TODO: student ID]

[TODO: full name] – [TODO: student ID]

[TODO: full name] – [TODO: student ID]

[TODO: full name] – [TODO: student ID]

Pham Tuan Anh – 2452081

Ho Chi Minh City, November 2026

Contents

1 Objectives **2**

2 Operating-system background **2**
2.1 Processes and threads (Chapters 3–4) 2
2.2 Multicore programming and Amdahl's law (Chapter 4) 2
2.3 CPU scheduling and context switching (Chapters 3 and 5) 3
2.4 Synchronization (Chapter 6) 3
2.5 Caches and memory (Chapters 1 and 8) 3
2.6 System calls and timing (Chapter 2) 3

3 Method **4**
3.1 Monte Carlo estimation of π 4
3.2 Measurement machine 4
3.3 Protocol 4

4 Design of the three approaches **5**
4.1 Approach 1 – single thread 5
4.2 Approach 2 – local accumulation (map–reduce) 5
4.3 Approach 3 – shared counter with a mutex 5
4.4 Correctness checks 6

5 Results **6**
5.1 Execution time, speedup and efficiency 6
5.2 Accuracy of π 8

6 Analysis **9**
6.1 A. Non-linear speedup 9
6.2 B. Saturation point 9
6.3 C. Oversubscription 9
6.4 D. Synchronization cost 10
6.5 E. Parallel efficiency 10

7 Trade-off experiments **10**
7.1 Synchronization frequency 10
7.2 Thread scaling 11
7.3 Local versus shared computation 12

8 Best practices identified **12**

9 Conclusion **12**

10 Team contributions **13**

Use of AI tools **13**

1 Objectives

The group estimates π with a Monte Carlo method in three ways: one thread, several threads that count locally, and several threads that share one counter protected by a mutex. The value of π is only the vehicle. What we measure and explain is:

- how much parallel execution improves performance, and where scalability stops;
- the cost of synchronization and communication between threads;
- which multi-threading design patterns are correct and efficient;
- how the observed behaviour follows from the OS scheduler and the hardware.

Source code, raw measurements and scripts: https://github.com/PTAxHVA/os-week3-monte-carlo-pi. Every change went through a pull request reviewed before merge.

2 Operating-system background

This section summarises the course material (CO2017 lecture slides, Silberschatz *et al.*, 10th ed.) that the experiment relies on. Each concept is tied to the place where it shows up in our measurements.

2.1 Processes and threads (Chapters 3–4)

A **process** is a program in execution with its own address space: code, data, heap and stack, plus a process control block (PCB) holding its state, registers and open files. A **thread** is the unit of CPU utilisation inside a process. Threads of one process share code, global data, heap and open files, but each thread has its own thread ID, program counter, register set and stack.

This split is exactly what the three approaches use:

- the global counter `g_inside` of Approach 3 lives in the shared data segment, so every thread sees the same variable and must synchronise;
- the counter `local_inside` of Approach 2 lives on each thread's private stack, so no other thread can touch it and no lock is needed.

Creating a thread is much cheaper than creating a process (no new address space), but it is not free; the creation and join of every thread is inside our timed interval.

We use **POSIX Pthreads** (`pthread_create`, `pthread_join`, `pthread_mutex_lock/unlock`). On Linux every Pthread is a kernel-scheduled task (one-to-one model), so the OS scheduler sees and places each of our threads individually.

2.2 Multicore programming and Amdahl's law (Chapter 4)

On a multicore system threads run in **parallel** only up to the number of cores; beyond that they run **concurrently**, interleaved on the same cores. The Monte Carlo loop is **data parallelism**: the same operation on different subsets of the points.

Amdahl's law bounds the speedup of a program whose fraction s is serial:

$$S(N) \le \frac{1}{s + (1-s)/N}, \qquad \lim_{N\to\infty} S(N) = \frac{1}{s}. \tag{1}$$

In our program the serial part is thread creation, join and the final reduction. A lock taken on every iteration effectively turns most of the loop into a serial part, which is why Approach 3 cannot speed up.

2.3 CPU scheduling and context switching (Chapters 3 and 5)

A thread is either running, ready or waiting. The **short-term scheduler** picks a ready thread for each core and the **dispatcher** performs the **context switch**: it saves the registers of the old thread and loads those of the new one. A context switch is pure overhead, and after it the new thread finds caches and the TLB filled with someone else's data.

Linux schedules threads with a fair, preemptive scheduler: when there are more runnable threads than cores, each thread gets a time slice in turn and the CPU time is split evenly. On a multiprocessor the scheduler also performs **load balancing** between per-core run queues and tries to keep **processor affinity** so that a thread stays near its warm cache. These mechanisms explain the saturation and oversubscription results (Sections 6.2–6.3).

A virtual machine adds one more level: the vCPUs of our measurement VM are themselves threads scheduled by the host OS.

2.4 Synchronization (Chapter 6)

A **race condition** occurs when several threads access shared data concurrently and the result depends on the order of execution. `g_inside++` looks atomic but compiles to load–add–store; two threads can both load the same old value and one increment is lost. The code that touches shared data is a **critical section**, and a correct solution must provide **mutual exclusion**, **progress** and **bounded waiting**.

A **mutex lock** provides mutual exclusion through `acquire()` (`pthread_mutex_lock`) and `release()` (`pthread_mutex_unlock`), built on atomic hardware instructions such as compare-and-swap. A **spinlock** busy-waits; the glibc mutex instead **blocks** under contention. On Linux it is a `futex`: the uncontended path is a single atomic instruction in user space, while a contended lock costs a system call to sleep and another to wake the waiter, each with a context switch. The cost of synchronization therefore depends less on the lock itself than on how often threads collide on it.

2.5 Caches and memory (Chapters 1 and 8)

Each core has private caches, and hardware keeps them **coherent**: a write to a cache line invalidates the copies in other cores. A variable written by all cores, such as the mutex and `g_inside`, keeps moving its cache line between cores. **False sharing** is the same effect on *different* variables that happen to share one 64-byte line, e.g. `count[0]` and `count[1]` of a shared array. This is why Approach 2 counts in a stack variable.

2.6 System calls and timing (Chapter 2)

Programs reach OS services through **system calls**. We time with `clock_gettime(CLOCK_MONOTONIC)`, which gives wall-clock time that never jumps backwards. We do not use `clock()`: it returns the CPU time summed over all threads, which grows with the thread count even when the wall-clock time falls. We also avoid `rand()`: glibc protects its single shared state with an internal lock, which would add hidden contention to every approach. Each thread instead uses `rand_r` with its own seed.

3 Method

3.1 Monte Carlo estimation of π

A point (x, y) is drawn uniformly from $[0, 1]^2$. It falls inside the quarter circle of radius 1 when $x^2 + y^2 \leq 1$, and the quarter circle covers $\pi/4$ of the square:

$$\hat{\pi} = 4 \cdot \frac{\text{points inside}}{\text{total points}}, \qquad \sigma_{\hat{\pi}} = 4\sqrt{\frac{p(1-p)}{n}} \approx \frac{1.642}{\sqrt{n}}, \quad p = \frac{\pi}{4}. \tag{2}$$

The assignment draws the picture on $[-1, 1]^2$ with a full circle; the area ratio is the same $\pi/4$. With $n = 10^7$ points the standard error is about 5.2×10^{-4}.

3.2 Measurement machine

All timing ran on one machine, as the instructor required (`results/machine.txt`).

Table 1: Measurement machine

Host	Laptop with AMD Ryzen 5 7235HS (4 cores, 8 hardware threads)
Measured system	VMware virtual machine, 4 vCPUs (1 socket × 4 cores, 1 thread per core), `nproc` = 4
OS	Ubuntu, Linux kernel 7.0.0-38-generic, x86_64
Compiler	gcc 15.2.0, `-O2 -Wall -pthread`
Date	2 November 2026, 16:11

A first run on a 2-vCPU VM showed only one step of scaling ($1 \to 2$ threads), so the VM was resized to 4 vCPUs and every measurement was repeated; this report uses only the 4-vCPU data. We did not give the VM all 8 host threads, so that the host OS keeps its own CPU time and does not compete with the measured vCPUs.

3.3 Protocol

Table 2: Measurement protocol (`scripts/run_bench.sh`)

Points per run	$n = 10^7$ for every configuration
Seed	42; thread `tid` uses seed `42 + tid`
Thread counts N	1, 2, 4, 8, 16, 32, 64, 128
Runs	5 recorded runs per configuration; Approaches 1 and 2 also run one discarded warm-up
Reported value	mean and standard deviation of the 5 runs
Timed interval	from just before the first `pthread_create` to just after the last `pthread_join` and the reduction
Not timed	argument parsing, process start-up, printing

Metrics: speedup $S(N) = T_{single}/T_{parallel}(N)$ and parallel efficiency $E(N) = S(N)/N$, where T_{single} is the mean time of Approach 1.

4 Design of the three approaches

All programs follow one contract (`src/common.h`): points in $[0,1]^2$ from `rand_r`; the remainder $n \bmod N$ goes to the first threads; output is one CSV line `approach,threads,k,points,inside,pi,time_sec`. Because Approaches 2 and 3 split the points and seed the threads identically, for the same N they draw *exactly the same points*, so their `inside` counts must be equal. This is our main correctness check.

4.1 Approach 1 – single thread

One loop draws n points and counts the hits; nothing is printed inside the loop. Its mean time is T_{single}.

4.2 Approach 2 – local accumulation (map–reduce)

Each thread gets its own argument struct, counts hits in a stack variable and writes the result back once. The main thread joins all threads and adds the partial counts.

```c
static void *worker(void *arg_ptr) {
    thread_arg_t *arg = (thread_arg_t *)arg_ptr;
    unsigned int seed = arg->seed;          /* private seed: base + tid */
    long long local_inside = 0;             /* on this thread's stack   */
    for (long long i = 0; i < arg->points; i++) {
        double x = rand01(&seed), y = rand01(&seed);
        if (x * x + y * y <= 1.0) local_inside++;
    }
    arg->inside = local_inside;             /* written back once        */
    return NULL;
}
```

Listing 1: Worker of Approach 2 (src/pi_local.c)

4.3 Approach 3 – shared counter with a mutex

The global `g_inside` is protected by a `pthread_mutex_t`. Each thread adds its hits to `g_inside` under the lock every k iterations, plus once for the remainder. With $k = 1$ (the standard Approach 3) the lock is taken on *every* iteration; larger k is the trade-off experiment of Section 7.1.

```c
static int add_batch(long long batch) {
    int rc = pthread_mutex_lock(&g_lock);
    if (rc != 0) return rc;
    g_inside += batch;                       /* critical section */
    return pthread_mutex_unlock(&g_lock);
}
/* inside worker(): */
for (long long i = 0; i < args->n_points; i++) {
    double x = rand01(&seed), y = rand01(&seed);
    if (x * x + y * y <= 1.0) batch++;
    if ((i + 1) % args->k == 0) { args->error = add_batch(batch); batch = 0; }
}
```

Listing 2: Critical section and worker loop of Approach 3 (src/pi_mutex.c)

4.4 Correctness checks

- For all 8 thread counts, Approach 2 and Approach 3 (every k) give the same `inside`; for $N = 1$ it equals Approach 1 (7 851 330). Checked over all 110 rows of `results/raw.csv`.
- With the lock removed from Approach 3 ($N = 4$, $k = 1$), three runs gave `inside` = 1 666 250, 1 707 553 and 1 746 813 instead of 7 853 800 ($\hat{\pi} \approx 0.67$–0.70): about 78% of the increments were lost to the race condition, and a different amount each run. This test build is not in the repository and ran on a separate 4-vCPU Ubuntu VM; it serves only as correctness evidence.

5 Results

5.1 Execution time, speedup and efficiency

$T_{single} = 0.1028\,\text{s}$ (mean of 5 runs, standard deviation 0.0008 s).

Table 3: Mean time of 5 runs, speedup and efficiency; mutex columns use $k = 1$

	Approach 2 – local			Approach 3 – mutex, $k = 1$		
N	T (s)	S	E	T (s)	S	E
1	0.1066	0.96	0.96	0.1285	0.80	0.80
2	0.0550	1.87	0.93	0.2670	0.38	0.19
4	**0.0287**	**3.58**	0.90	0.2838	0.36	0.09
8	0.0294	3.49	0.44	0.2903	0.35	0.044
16	0.0303	3.40	0.21	0.2977	0.35	0.022
32	0.0315	3.26	0.10	0.2959	0.35	0.011
64	0.0465	2.21	0.034	0.2964	0.35	0.005
128	0.0433	2.37	0.019	0.2976	0.35	0.003

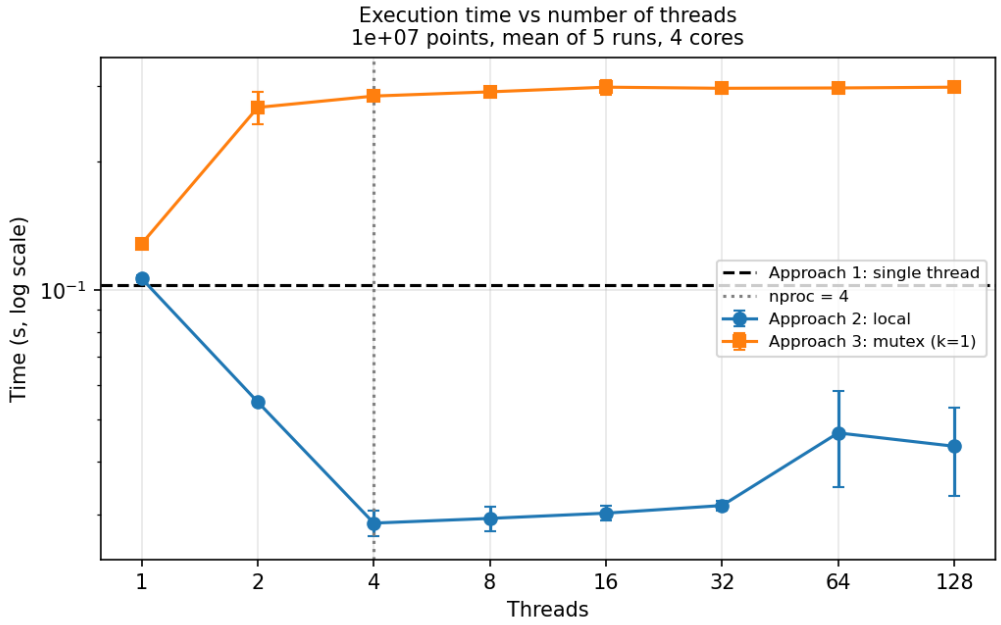


Figure 1: Execution time vs number of threads (log scale; error bars are one standard deviation)

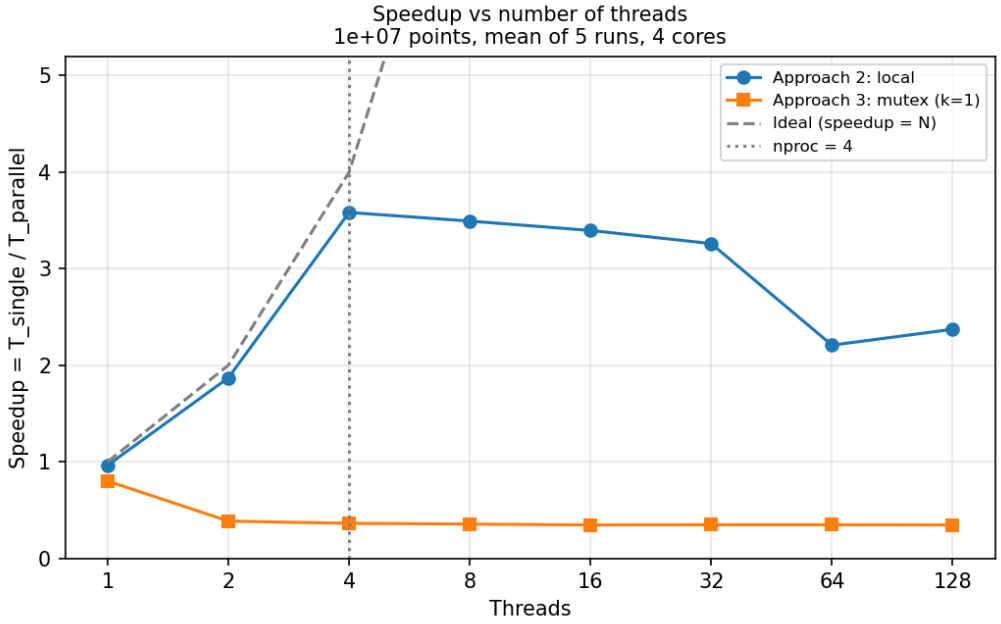


Figure 2: Speedup vs number of threads, against the ideal $S = N$

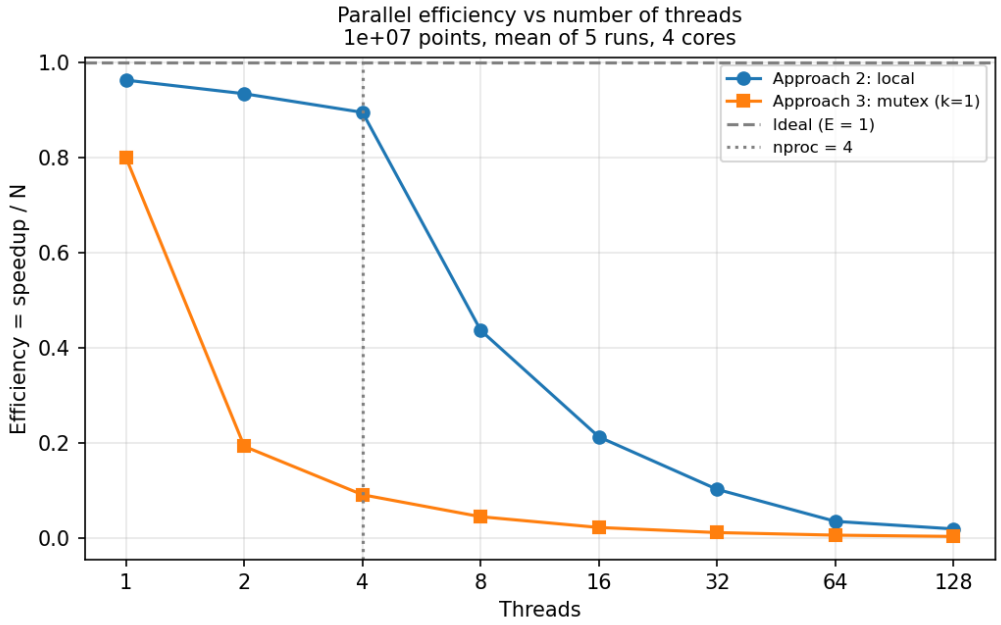


Figure 3: Parallel efficiency $E = S/N$

5.2 Accuracy of π

In the timing runs all 5 runs of a configuration use seed 42, so they do the same work and give the same π; only the time varies. To show how π varies between runs, `scripts/run_precision.sh` adds 5 runs with 5 different seeds, 1000 apart. Seeds must be far apart because thread `tid` uses seed `seed + tid`: two runs whose seeds differ by 1 would reuse 3 of the 4 random streams.

Table 4: π over 5 runs with different seeds, $n = 10^7$, $N = 4$ (mutex with $k = 1000$)

Run	Seed	Single thread	Local	Mutex
1	42	3.140532	3.141520	3.141520
2	1042	3.142388	3.141289	3.141289
3	2042	3.140790	3.141713	3.141713
4	3042	3.142003	3.141425	3.141425
5	4042	3.141239	3.141278	3.141278
Mean		3.141390	3.141445	3.141445
Standard deviation		0.00079	0.00018	0.00018
$\lvert\bar{\pi} - \pi\rvert$		0.00020	0.00015	0.00015

All values lie in 3.1403–3.1424, consistent with the standard error of (2). The mutex version matches the local version digit for digit in all 5 runs. glibc's `rand_r` produces the same sequence on every machine, so these π values do not depend on the measurement machine.

Five to six correct decimals. By (2), each extra correct digit needs 100 times more points: 5 decimals (error $\sim 3 \times 10^{-6}$) need about $3\text{–}5 \times 10^{11}$ points, 6 decimals about 4×10^{13}. `rand_r` cannot do this. Its state is 32 bits, so its sequence repeats after at most 2^{32} numbers, about 2×10^9 points. A χ^2 test on 10^7 consecutive (x, y) pairs in a 100×100 grid also gives 8 588, well

below the expected 95% range [9 724; 10 278]: the points of this linear congruential generator are spread *more evenly* than true random points. The one-dimensional distribution passes ($\chi^2 = 925.5$ with 999 degrees of freedom), and the streams of seeds 42 and 43 have correlation -0.0003.

We therefore added `src/pi_big.c`: the same structure as Approach 2, but each thread uses the 64-bit xoshiro256+ generator (period $2^{256} - 1$, seeded through splitmix64 so that threads get distinct streams), 53-bit doubles and 64-bit counters.

Table 5: `pi_big`, $N = 4$, seed 42 (`results/precision.csv`)

n	$\hat{\pi}$	$\lvert\hat{\pi} - \pi\rvert$	$\sigma_{\hat{\pi}}$
10^7	3.1419652	3.7×10^{-4}	5.2×10^{-4}
10^8	3.1416643	7.2×10^{-5}	1.6×10^{-4}
10^9	3.1415584	3.4×10^{-5}	5.2×10^{-5}
10^{10}	3.1415802	1.2×10^{-5}	1.6×10^{-5}
10^{11}	**3.1415946**	2.0×10^{-6}	5.2×10^{-6}
5×10^{11}	**3.1415960**	3.3×10^{-6}	2.3×10^{-6}

From 10^{11} points on, the estimate has 5 correct decimals, 3.14159. The 5×10^{11}-point run took **[TODO: time_sec of the last row of results/precision.csv]** s on the 4-vCPU measurement VM. Six decimals would need about 4×10^{13} points, 80 times the largest run above, i.e. several hours; we did not run it. This is the inherent limit of Monte Carlo: the error falls only as $1/\sqrt{n}$.

6 Analysis

6.1 A. Non-linear speedup

The measured speedups are $S(2) = 1.87$ and $S(4) = 3.58$, below the ideal 2 and 4. Solving Amdahl's law (1) for $S(4) = 3.58$ gives an apparent serial fraction $s \approx 3.9\%$. This is more than the real serial part of the program (thread creation, join, reduction), because the fitted s also absorbs costs that Amdahl's law does not model: the vCPUs are scheduled by the host, so a vCPU is sometimes paused; the CPU clock may drop when several cores are busy; and the join waits for the slowest thread. Even at $N = 1$, Approach 2 is about 4% slower than Approach 1 (0.1066 vs 0.1028 s), the cost of creating and joining one thread.

6.2 B. Saturation point

Threads run truly in parallel only up to the number of cores the OS can schedule on, here `nproc` = 4. The best time of Approach 2, 0.0287 s, is at $N = 4$, exactly the vCPU count. From 4 to 8, 16 and 32 threads the time barely changes (0.0287 → 0.0315 s): extra threads add no cores, they only share the 4 cores by time slicing. The curve has the shape of the sample chart in the assignment (4-core machine, peak at 4 threads).

6.3 C. Oversubscription

With more threads than cores, the scheduler gives each ready thread a time slice in turn. Total CPU time stays the same, so useful work per second does not increase, while every extra thread adds cost:

- a context switch each time the scheduler swaps threads on a core;
- caches, TLB and branch predictors partly refilled after each switch;
- creation and join time per thread;
- uneven finishing: the join waits for the last thread, so a thread descheduled near the end delays the whole run.

The data show this: 32 threads take 0.0315 s, 64 threads 0.0465 s, 62% slower than $N = 4$. The standard deviation also jumps, from at most 0.002 s for $N \leq 32$ to 0.012 s at $N = 64$ and 0.010 s at $N = 128$: the run time now depends on the scheduling order, which differs from run to run. $N = 128$ is slightly faster than $N = 64$ (0.0433 vs 0.0465 s), within the standard deviation of both points, so we draw no conclusion from it.

6.4 D. Synchronization cost

The increment of `g_inside` is a critical section. With $k = 1$ every iteration needs the lock:

- **Contention.** A free futex-based mutex costs one atomic instruction; a held one sends the caller to sleep in the kernel and the releasing thread must make another system call to wake it. Each sleep and wake-up is a context switch, far more expensive than the few arithmetic operations of one Monte Carlo sample.
- **Serialization.** However many threads run, the increments happen one after another, so the part of the loop that matters cannot speed up (Amdahl with s close to 1).
- **Cache-line transfer.** The mutex and the counter are written by every core, so their cache line keeps moving between cores.

At $N = 1$ (no contention) lock and unlock alone cost $0.1285 - 0.1028 = 0.026$ s for 10^7 iterations, about 2.6 ns per pair. At $N = 4$ Approach 3 takes 0.2838 s against 0.0287 s for Approach 2: **9.9 times slower**, and almost 3 times slower than the single thread. From $N = 2$ on the time is almost constant (0.27–0.30 s): more threads only add more waiters for the same lock.

6.5 E. Parallel efficiency

$E(N)$ tells how much of each added thread's capacity turns into useful speedup. For Approach 2 it stays at 0.90–0.96 for $N \leq 4$ (each thread has its own core), then falls roughly as $4/N$ because S stops growing: 0.44 at $N = 8$, 0.10 at $N = 32$, 0.019 at $N = 128$. For Approach 3 the lock drops E from the second thread on: 0.19 at $N = 2$, 0.09 at $N = 4$.

7 Trade-off experiments

7.1 Synchronization frequency

Approach 3 was run with $N = 4$ and $k = 1$, 10, 100, 1 000, 10 000, 100 000. Each thread now takes the lock about $n/(Nk)$ times instead of once per point, so the contention cost shrinks with k.

Table 6: Approach 3 at $N = 4$ versus k; Approach 2 at the same N takes 0.0287 s

k	Lock acquisitions per thread	T (s)	Relative to Approach 2
1	2 500 000	0.2838	×9.9
10	250 000	0.0800	×2.8
100	25 000	0.0330	+15%
1 000	2 500	0.0318	+11%
10 000	250	0.0318	+11%
100 000	25	0.0304	+6%

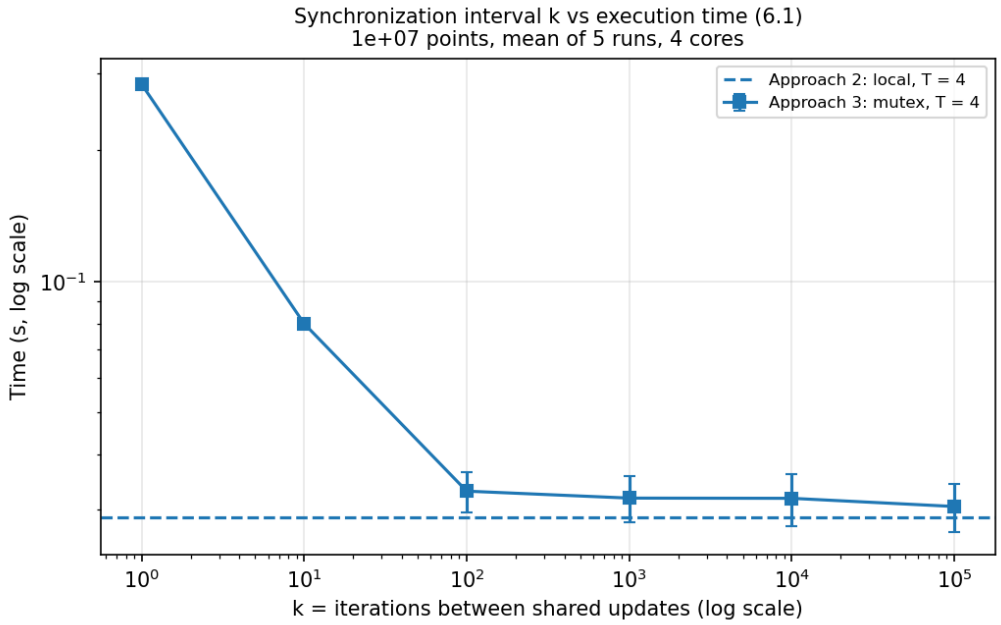


Figure 4: Approach 3 time vs synchronization interval k ($N = 4$); the dashed line is Approach 2

From $k = 100$ on, the time is within 6–15% of Approach 2, of the order of the standard deviation of these points (0.003–0.004 s). The rest is the modulo `(i + 1) % k` executed on every iteration, which Approach 2 does not have. Synchronising rarely enough removes almost all of the lock cost.

7.2 Thread scaling

The thread counts cover three regions: fewer threads than cores ($N < 4$), equal ($N = 4$) and more, up to 128. Figures 1 and 2 show the expected shape: time falls until $N = 4$, stays flat until $N = 32$, then rises at $N = 64$–128 because of oversubscription (Section 6.3).

7.3 Local versus shared computation

Table 7: Approach 2 versus Approach 3 ($k = 1$)

	Approach 2 (local)	Approach 3 (shared, $k = 1$)
Critical section	none in the loop; only the reduction after join	the increment, once per iteration
Contention	none	every thread competes for one mutex
Serialization	none during computation	all increments one at a time
Time at $N = 4$	0.0287 s	0.2838 s

8 Best practices identified

Each practice is backed by one of our measurements.

1. **Accumulate locally, combine once (map–reduce).** At $N = 4$: 0.0287 s versus 0.2838 s with a locked shared counter.
2. **Match the thread count to the cores.** Speedup stops at $N = 4 =$ `nproc`; far too many threads ($N = 64$) are 62% slower.
3. **Synchronize as rarely as possible.** With $k \geq 100$ the mutex version is within 6–15% of the lock-free one.
4. **Give each thread its own random generator state.** `rand_r` with a private seed; `rand()` would serialize threads behind a hidden lock.
5. **Keep per-thread data off shared cache lines.** Count in a stack variable, not in `count[tid]` (false sharing).
6. **Measure wall-clock time correctly.** `CLOCK_MONOTONIC`, not `clock()`; repeat runs and report mean and spread. At $N \geq 64$ the standard deviation is 5 times larger, so a single run would mislead.
7. **Check correctness, not only speed.** The same seed must give the same `inside` in every approach; removing the lock lost about 78% of the increments.
8. **Pick a random generator good enough for the accuracy you need.** 32-bit `rand_r` repeats after $\sim 2 \times 10^9$ points; 5 correct decimals need a 64-bit generator and $\geq 10^{11}$ points.

9 Conclusion

The almost fully parallel estimator reaches a speedup of 3.58 on 4 vCPUs, saturates exactly at the core count and slows down under oversubscription. A mutex taken on every iteration makes the program slower than a single thread; synchronising every 100 iterations or more removes nearly all of that cost. For accuracy, 10^7 points give about 3 correct decimals of π; 5 decimals need 10^{11} points and a 64-bit random generator.

10 Team contributions

Member	Work	Share
[TODO: name] (Task 1)	`common.h` contract, Approach 1, `Makefile`, README, integration and submission	20%
[TODO: name] (Task 2)	Approach 2 (local accumulation); analysis A, B, C and thread scaling	20%
[TODO: name] (Task 3)	Approach 3 (mutex, parameter k); analysis D, E, synchronization frequency, local vs shared	20%
[TODO: name] (Task 4)	All measurements on the shared machine, results and figures	20%
Pham Tuan Anh (Task 5)	Review of every pull request, benchmark and plotting scripts, accuracy experiment, report	20%

Use of AI tools

As required by the course syllabus: **[TODO: the group lists the AI tools used and what for, e.g. Claude Code for task planning, benchmark scripts, pull-request review and drafting this report]**. All numbers in this report come from running the group's code.